# Markov Decision Process (MDP) no Contexto de Reinforcement Learning

**Autor:** Marco Aurélio Guimarães Falcão – Engenheiro de Sistemas
**Data:** 30/10/2026

## Resumo

Este artigo apresenta o formalismo dos Processos de Decisão de Markov (MDP) no contexto do Aprendizado por Reforço, contemplando ambientes com múltiplos estados e dinâmica de transição. São apresentados a cadeia de Markov, a definição formal de MDP, a política, o retorno, as funções de valor e as equações de Bellman, nas formas escalar, vetorizada e ótima. Em seguida, são descritos os métodos de solução por Iteração de Valor, Iteração de Política, Generalized Policy Iteration e Modified Policy Iteration, com a comparação de seus custos computacionais. Os algoritmos foram implementados em Python, com modelagem orientada a objetos e padrões de projeto, e aplicados a quatro problemas: um robô logístico em tabuleiro 2 x 2, o tabuleiro 5 x 5 do livro de Sutton e Barto, o Lago Congelado (Frozen Lake) e uma versão simplificada do jogo do Wumpus. Os resultados confirmaram o funcionamento correto dos algoritmos e evidenciaram as limitações dos MDP, em especial a necessidade de conhecer o modelo do ambiente e o crescimento do custo computacional com o número de estados.

**Palavras-chave:** Aprendizado por Reforço; Processos de Decisão de Markov; Equações de Bellman; Iteração de Valor; Iteração de Política; Programação Dinâmica.

## 1. Introdução

Como ilustrado na Figura 1, os conceitos fundamentais do Aprendizado por Reforço podem ser apresentados a partir da interação simplificada entre agente e ambiente explorada no estudo do problema dos $K$-bandidos armados (Referência [4]). Naquele cenário, o problema é puramente **não-associativo**: assume-se a existência de um **único estado estático** (ou ambiente invariante), no qual o objetivo do agente reduz-se a mapear ações diretamente a recompensas imediatas, sem a necessidade de avaliar como a ação presente condiciona a evolução futura do estado do sistema.

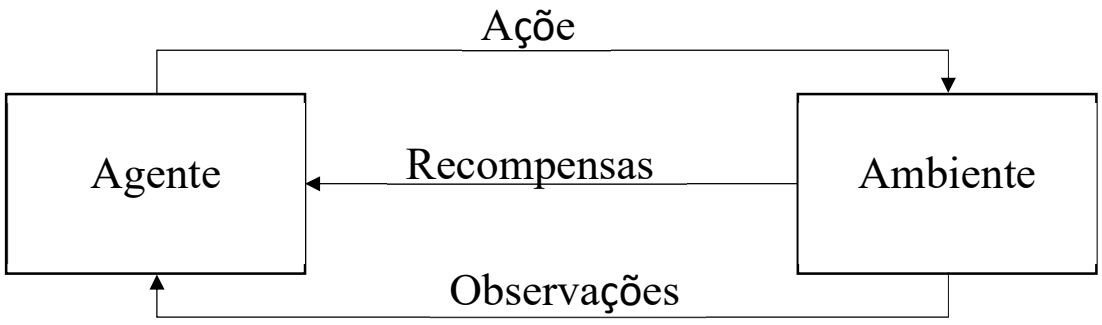



**Figura 1** – Esquema Geral do Aprendizado por Reforço

Entretanto, na vasta maioria dos problemas reais de tomada de decisão sequencial, o ambiente exibe dinâmica complexa e múltiplos estados, ocorrendo dessa forma:

1. **Dependência do Estado:** A recompensa esperada $R_{t+1}$ associada à escolha de uma mesma ação $a$ varia substancialmente a depender do estado corrente $s$ em que o agente se encontra.
2. **Transição Dinâmica (Aprendizado Associativo):** A ação $A_t$ executada pelo agente não apenas gera um sinal de recompensa imediato, mas altera a própria trajetória do sistema, conduzindo o ambiente a um novo estado $S_{t+1}$.

O presente artigo estende a formulação dos $K$-bandidos ao adotar o formalismo dos **Processos de Decisão de Markov (MDP)**. O MDP provê o arcabouço matemático rigoroso capaz de unificar percepção de estado, controle de ações, transições probabilísticas e otimização de recompensas acumuladas ao longo do tempo em ambientes multiestados sob incerteza.

Este estudo foi conduzido pelo autor com o suporte de ferramentas de Inteligência Artificial — **Gemini, Claude e DeepSeek** — operando no ambiente de desenvolvimento **Google Colab**. A colaboração com as IAs abrangeu etapas cruciais da pesquisa, incluindo a revisão textual, o levantamento bibliográfico, a derivação das equações matemáticas do Processo de Decisão de Markov (MDP), a modelagem orientada a objetos com aplicação de padrões de projeto em Python, a implementação, teste e depuração dos algoritmos, além da análise dos resultados e da discussão das conclusões.

## 2. Apresentação Geral de MDP (*Markov Decision Process*) no Contexto de Reinforcement Learning e Definição do Problema.

### 2.1 – Cadeia de Markov

Uma **Cadeia de Markov** (ou *Markov Chain*) representa um sistema estocástico totalmente autônomo e passivo. Nela, as transições entre estados ocorrem de forma espontânea, impulsionadas unicamente pela dinâmica interna do ambiente, sem a interferência de ações externas ou a atribuição de recompensas. A cadeia de Markov é definida apenas pela sua Matriz de Transição de Estados $P(s'|s)$, uma matriz n x n, no qual n é o número de estados.

Em uma cadeia de Markov, podemos observar uma sequência de estados:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \cdots$$

Um **episódio** é uma trajetória do processo que começa em um estado inicial e termina quando o processo alcança uma condição de término.

Por exemplo, em um jogo:

$$\text{Início} \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow \text{Vitória}$$

ou

$$\text{Início} \rightarrow S_1 \rightarrow S_2 \rightarrow \text{Derrota}$$

O episódio é toda a sequência entre o início e o término.

Um episódio é **finito** quando existe um instante $T < \infty$ em que o processo termina:

$$S_0, S_1, \dots, S_T$$

e $S_T$é um **estado terminal**.

Depois disso, não há mais decisões ou transições relevantes para aquele episódio.

Um exemplo simples, onde $D$é terminal:

$$A \rightarrow B \rightarrow C \rightarrow D$$

Podemos representar:

$$P(D \mid D) = 1$$

Ou seja, uma vez que chegamos a $D$, permanecemos em $D$. Este modelo é conhecido como de horizonte finito [3].

Um episódio é **infinito** quando não existe um instante terminal finito:

$$S_0, S_1, S_2, \dots$$

A sequência continua indefinidamente.

Por exemplo:

$$A \rightarrow B \rightarrow A \rightarrow B \rightarrow A \rightarrow \cdots$$

Não existe um estado $T$ tal que o processo termine.

Isso aparece naturalmente em problemas como controle de temperatura, operação de uma máquina, mercado financeiro etc., nos quais não existe necessariamente um "fim do jogo". Este modelo é conhecido como de horizonte infinito [3].

Podemos **transformar matematicamente o episódio finito em uma sequência infinita**, fazendo o estado terminal absorver o processo:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow D \rightarrow D \rightarrow \cdots$$

Assim, a cadeia passa a ter infinitos instantes:

$$S_0, S_1, S_2, \dots$$

mas, depois de atingir $D$, ela permanece eternamente nesse estado.

A **dinâmica de transição** de uma Cadeia de Markov homogênea é totalmente caracterizada pela sua Matriz de Transição de Estados $P(s' \mid s)$.

> **Obs:** O termo **homogênea** (ou *estacionária no tempo*) para uma Cadeia de Markov significa que as probabilidades de transição entre os estados não mudam ao longo do tempo.

A Figura 2 ilustra a aplicação prática de uma Cadeia de Markov para a modelagem da negociação mensal do ativo ITUB4 na Bolsa de Valores no período de 2012 a 2025 (Referência [5]). Nesse modelo, a dinâmica do papel é discretizada em um espaço de estados definido pela direção da variação mensal do preço, dividindo-se entre os estados de **Alta** (dia com fechamento positivo) e **Queda** (dia com fechamento negativo).

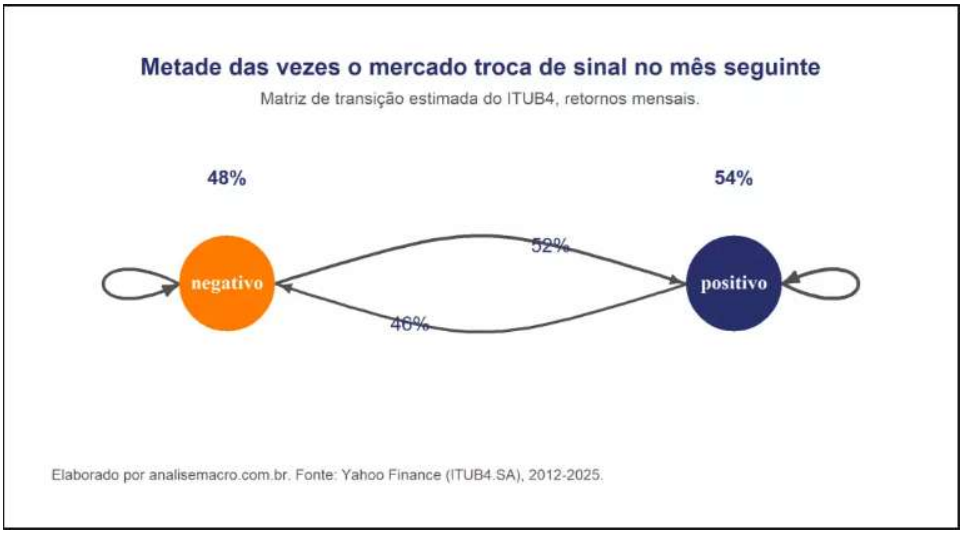



**Figura 2** – Exemplo de Cadeia de Markov

> **Obs:** Note que temos um episódio infinito pois não há estado terminal (modelo de horizonte infinito).

O ponto crucial dessa abordagem reside no fato de que essas percentagens de alternância de regimes não derivam da intervenção de um único investidor, mas sim da própria microestrutura e do comportamento estocástico inerente ao mercado. Na ausência de um vetor de ações capaz de alterar as probabilidades de transição e sem a contabilização explícita de sinais de recompensa, o sistema opera estritamente como um **observador passivo** da frequência temporal dos eventos. Essa caracterização define rigorosamente um **Processo de Markov** (ou Cadeia de Markov), no qual a evolução do sistema é descrita apenas por probabilidades condicionais de estado para estado.

## 2.2 – Processo de Decisão de Markov (MDP)

Em um **Processo de Decisão de Markov (MDP)**, o dinamismo do sistema é profundamente retratado a exemplo da Figura 1 como uma interação malha-fechada: as transições de estado passam a ser **disparadas e condicionadas à execução de ações** selecionadas ativamente por um agente. Além disso, cada transição gera uma recompensa escalar $R_{t+1}$, que avalia a qualidade da decisão tomada.

No arcabouço formal do MDP, tanto a trajetória dos estados futuros $S_{t+1}$ quanto a magnitude das recompensas $R_{t+1}$ são modeladas como **processos estocásticos acoplados**, cujas distribuições de probabilidade dependem do par estado-ação atual $(S_t, A_t)$, conforme formalizado pela distribuição conjunta $p(s', r \mid s, a)$.

A dinâmica completa do ambiente em um MDP (Processo de Decisão de Markov) é inteiramente especificada por uma **função de distribuição de probabilidade conjunta condicional**, denotada por $p: \mathcal{S} \times \mathbb{R} \times \mathcal{S} \times \mathcal{A} \to [0,1]$. Essa função mapeia a probabilidade de transição para um novo estado $s' \in \mathcal{S}$ acompanhado de uma recompensa escalar $r \in \mathcal{R} \subset \mathbb{R}$, dado o estado atual $s \in \mathcal{S}$ e a ação executada $a \in \mathcal{A}$:

$$p(s', r \mid s, a) \doteq \mathbb{P}(S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a) \quad (1)$$

Um **Processo de Decisão de Markov (MDP)** é a estrutura matemática formal utilizada para modelar a tomada de decisão sequencial em ambientes sob incerteza, fundamentando a teoria de Aprendizado por Reforço (RL). Ele é rigorosamente definido pela tupla 5-dimensional $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, na qual:

- $\mathcal{S}$: representa o espaço de estados finito, $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$;
- $\mathcal{A}$: representa o espaço de ações finito, $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$;
- $\mathcal{P}$: é a dinâmica do **Modelo de Transição de Estados**, dada pela probabilidade condicional:

$$\mathcal{P}(s' \mid s, a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a) \quad (2)$$

> **Obs:** Esta função de distribuição de probabilidades condicional mapeia a transição do par estado-ação corrente para o próximo estado $s'$, respeitando a **Propriedade Markoviana**: o próximo estado depende exclusivamente do estado e ação atuais, sendo independente do histórico passado.

- $\mathcal{R}$: é a função de recompensa imediata, tipicamente definida como $\mathcal{R}(s, a, s')$, que quantifica o sinal de feedback do ambiente para o agente, valor esperado ou média da recompensa ao executar a ação a no estado s, em transição para o estado s', em um **Modelo de Recompensas Esperadas**:

$$\mathcal{R}(s, a, s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] \quad (3)$$

- $\gamma \in [0,1)$: é o fator de desconto (*discount factor*), que pondera o valor presente das recompensas obtidas no futuro.

A probabilidade condicional conjunta $p(s', r \mid s, a)$ é a representação completa de um Processo de Decisão de Markov (MDP). A partir dela, consegue-se derivar por **marginalização** tanto o **modelo de transição de estados** quanto **o modelo de recompensas esperadas**.

Para obter o **Modelo de Transição de Estados** por marginalização soma-se todas as recompensas possíveis $r$:

$$P(s' \mid s, a) = \sum_{r \in \mathcal{R}} p\,(s', r \mid s, a) \tag{4}$$

Para obter o **Modelo de Recompensas Esperadas** calcula-se a média ponderada das recompensas $r$:

$$R(s, a, s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r \mid s, a)}{P(s' \mid s, a)} \tag{5}$$

A Figura 3 apresenta a dinâmica de um MDP para um jogo de Tabuleiro 2 x 2:

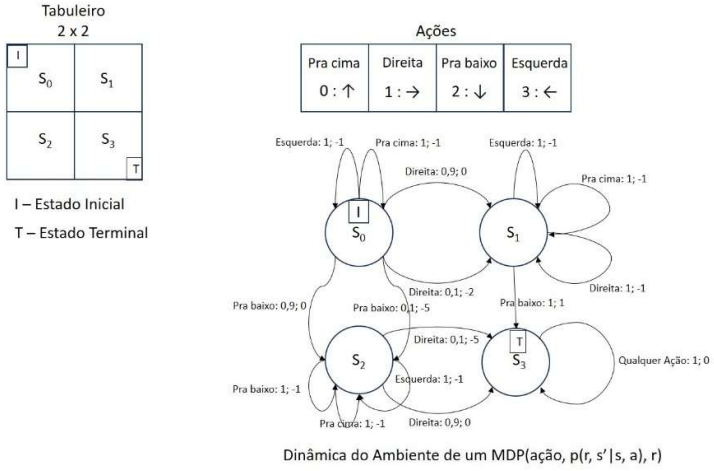



**Figura 3** – Dinâmica de MDP de um Tabuleiro

Este tabuleiro $2 \times 2$ modela duas trajetórias possíveis ($S_0 \rightarrow S_1 \rightarrow S_3$ e $S_0 \rightarrow S_2 \rightarrow S_3$) para um robô logístico encarregado de coletar peças no almoxarifado (estado $S_0$) e entregá-las no setor de produção (estado $S_3$). As transições apresentam recompensas estocásticas, simulando a ocorrência eventual de congestionamentos na linha de tráfego dos robôs.

O pacote para simulação de ambientes ***Gymnasium*** em Python [2] padroniza uma estrutura de dicionário para definição da Dinâmica do MDP, que é aplicada para o Tabuleiro descrito anteriormente, e mostrado na Figura 4. Essa estrutura expressa a probabilidade conjunta condicionada P(s', r | s, a) que define a Dinâmica do ambiente do MDP.



```
Python

# P[s][a] = [(prob, s_linha, recompensa, done), ...]
P = {
    0: {  # s0
        0: [(1.0, 0, -1, False)],                          # Cima
        1: [(0.9, 1, 0, False), (0.1, 1, -2, False)],      # Direita
        2: [(0.9, 2, 0, False), (0.1, 2, -5, False)],      # Baixo
        3: [(1.0, 0, -1, False)]                           # Esquerda
    },
    1: {  # s1
        0: [(1.0, 1, -1, False)],                          # Cima
        1: [(1.0, 1, -1, False)],                          # Direita
        2: [(1.0, 3, 1, True)],                            # Baixo (s3 é estado ter
        3: [(1.0, 1, -1, False)]                           # Esquerda
    },
    2: {  # s2
        0: [(1.0, 2, -1, False)],                          # Cima
        1: [(0.9, 3, 0, True), (0.1, 3, -5, True)],       # Direita
        2: [(1.0, 2, -1, False)],                          # Baixo
        3: [(1.0, 2, -1, False)]                           # Esquerda
    },
    3: {  # s3 (Terminal)
        0: [(1.0, 3, 0, True)],                            # Cima
        1: [(1.0, 3, 0, True)],                            # Direita
        2: [(1.0, 3, 0, True)],                            # Baixo
        3: [(1.0, 3, 0, True)]                             # Esquerda
    }
}
```

**Figura 4** – Dicionário Padrão Gymnasium para Dinâmica do MDP

Ressalta-se que o ambiente ***FrozenLake*** do pacote **Gymnasium** adota uma codificação distinta para o espaço de ações (0: Esquerda, 1: Para baixo, 2: Direita e 3: Para cima) em relação àquela utilizada nas Figuras 3 e 4. Desse modo, é necessária a conversão direta dos índices ao integrar a dinâmica desse ambiente.

A estrutura do dicionário da Figura 4 representa o Modelo de Dinâmica do MDP em dois níveis de chaveamento: a primeira chave corresponde ao estado de origem $s$, enquanto a segunda chave representa a ação executada $a$. Para cada par $(s, a)$, o dicionário armazena uma lista de tuplas da forma (prob, próximo estado, recompensa, booleano), correspondendo respectivamente a $(p(s', r \mid s, a)$, $s'$, $r$, is_terminal). O booleano is_terminal indica se a transição vai para um estado terminal ou não. A soma das probabilidades $p(s', r \mid s, a)$ de todas as tuplas associadas a um mesmo par $(s, a)$ é estritamente unitária ($\sum_i \text{prob}_i = 1.0$), garantindo uma distribuição de probabilidade válida sobre os resultados estocásticos da ação.

## 2.3 – Política de um Processo de Decisão de Markov (MDP)

Em um Processo de Decisão de Markov (MDP), a **política $\boldsymbol{\pi(a|s)}$** representa uma distribuição de probabilidade sobre o conjunto de ações $A = \{a_1, a_2, \dots, a_m\}$ que o agente pode escolher quando se encontra em um determinado estado $s$.

Para que seja uma distribuição de probabilidade válida, duas condições fundamentais devem ser satisfeitas:

1. **Não-negatividade:** A probabilidade de escolher qualquer ação individual não pode ser negativa:

$$p(a_i|s) \geq 0 \quad \forall i \in \{1, \dots, m\} \tag{6}$$

2. **Soma Unitária:** A soma das probabilidades de todas as ações possíveis no estado $s$ deve ser exatamente igual a 1:

$$\sum_{i=1}^{m} p\,(a_i|s) = 1 \tag{7}$$

Existem os seguintes tipos de política em um MDP:

- ***Stochastic Policy*** **(Estocástica):** O vetor $[p(a_1|s), p(a_2|s), \dots, p(a_m|s)]$ atribui valores contínuos entre 0 e 1 para as ações, permitindo exploração e modelagem de incerteza (ex.: $\pi(a_1|s) = 0.7, \pi(a_2|s) = 0.3$).
- ***Deterministic Policy*** **(Determinística):** É um caso particular em que uma única ação $a_k$ recebe probabilidade 1 e todas as outras recebem 0 ($\pi(a_k|s) = 1$). Expressa-se diretamente como a função $a = \pi(s)$.

A **Matriz de Transição de Estados** induzida — $P^{\pi}(s' \mid s)\ para\ um\ MDP$ resulta da combinação do modelo de transição $P(s' \mid s, a)$ com uma política específica $\pi(a \mid s)$:

$$P^{\pi}(s' \mid s) = \sum_{a \in \mathcal{A}} \pi\,(a \mid s) P(s' \mid s, a) \tag{8}$$

Sendo assim representa a dinâmica da Cadeia de Markov resultante do comportamento do agente (**depende estritamente da política $\boldsymbol{\pi}$**).

No contexto do problema do robô logístico em um tabuleiro $2 \times 2$ da Figura 3, a operação da Equação (8) combina a matriz de política $\mathbf{\Pi}$ de dimensões $4 \times 4$ (na qual cada linha representa um estado $s$ e cada coluna uma ação $a$, tal que $\pi(a \mid s) = \mathbf{\Pi}_{s,a}$) com o tensor de transição do ambiente $\mathbf{P}$ de dimensões $4 \times 4 \times 4$, correspondente a $P(s, a, s') = P(s' \mid s, a)$.

A implementação computacional eficiente desta operação em Python é realizada por meio da função **np.einsum**, utilizando a notação de somatório de Einstein:

```
P_estados = np.einsum('sa, sad -> sd', PI, P).
```

Para um MDP com n estados e m ações em cada estado o número de políticas determinísticas possíveis é dado por $m^n$. A seção 3 irá tratar de algoritmos que solucionem o problema de encontrar uma política ótima para o MDP.

## 2.4 – Retorno, função de valor de estado e função de ação-valor

Para avaliar a qualidade das decisões a longo prazo, define-se o **Retorno Descontado** $G_t$ no instante temporal $t$ como a soma estocástica acoplada pelo fator de desconto para cada episódio:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \tag{9}$$

O fator $\gamma$ possui dupla finalidade: do ponto de vista matemático, garante a convergência da série em horizontes infinitos; e do ponto de vista conceitual, reflete a preferência temporal do agente (as recompensas mais recentes tem peso maior na computo de $G_t$).

A partir do retorno $G_t$, a avaliação de uma política $\pi(a|s)$ desdobra-se em duas funções fundamentais: A função valor do estado $V_\pi(s)$ e a função valor do par estado-ação $Q^\pi(s,a)$.

A **função de valor do estado** (*state value function*) para uma política $\pi(a \mid s)$ é definida como o valor esperado do retorno descontado $G_t$ dado o estado em que se encontra $S_t = s$:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[\textstyle\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s] \tag{10}$$

A **função de valor de estado-ação** (*action-state value function*) para uma política $\pi(a \mid s)$ é definida como o valor esperado do retorno descontado $G_t$ dado o estado em que se encontra $S_t = s$ e a ação executada nesse estado $A_t = a$.

$$\begin{aligned} &Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_\pi[\textstyle\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a] \end{aligned} \tag{11}$$

As duas funções de valor se relacionam da seguinte forma:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, Q^\pi(s,a) \tag{12}$$

$V^\pi(s)$é a **média ponderada** dos Q-valores, na qual os pesos são as probabilidades de escolher cada ação a em cada estado s.

> **Obs:** As funções de $valor\ de\ estado\ V(s)$ e de estado-ação $Q(s,a)$ são definidas *estritamente em relação a uma política específica* $\pi(a/s)$. Sem fixar ou assumir uma política $\pi$, o valor esperado dos retornos futuros não pode ser calculado, pois não haveria como saber com qual probabilidade o agente escolheria as ações nos estados subsequentes.

## 2.5 – Política Ótima

No contexto dos Processos de Decisão de Markov (MDP), a **política ótima ($\pi^*$)** define a estratégia ideal de tomada de decisão. O seu objetivo principal é orientar as ações do agente a cada estado de modo a maximizar a expectativa do retorno acumulado descontado ao longo de sua trajetória no ambiente.

Uma política $\pi^*$ é ótima se:

$$V^{\pi^*}(s) \geq V^\pi(s)\ \forall s, \forall \pi \tag{13}$$

Ou seja, **em todo estado**, o valor da política ótima é maior ou igual ao de qualquer outra política. Na próxima seção veremos alternativas de solução para obtenção da politica ótima para um MDP.

# 3. Estratégias de Solução:

## 3.1 – Equações de Bellman para as funções de valor do estado e de ação-estado

O problema de encontrar uma solução ótima para decisões sequenciais com modelo conhecido (***Model Based***) foi amplamente estudado por Bellman [6], que denominou essa abordagem como **Programação Dinâmica**. A ideia central baseia-se em abordar o problema de forma recursiva e reaproveitar os resultados parciais armazenados em memória para obter a convergência rumo à política ótima de ações, conforme formulado nos Processos de Decisão de Markov (MDP). O termo ***programação*** refere-se ao planejamento do encadeamento de decisões no tempo, enquanto ***dinâmica*** expressa o caráter evolutivo e adaptativo dos cálculos recursivos. A partir dessa formulação, deduz-se a **Equação de Bellman**, que se aplica tanto ao cálculo da função de valor do estado $V(s)$ quanto à função de estado-ação $Q(s,a)$.

***Equação de Bellman para*** $\boldsymbol{V^{\pi}(s)}$

Partimos da definição de $V^{\pi}(s)$ (equação (10)) para obter a Equação de Bellman para a função de valor do estado:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s]$$

Podemos fatorar o primeiro termo:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_{\pi}[R_{t+2} + \gamma R_{t+3} + \cdots \mid S_t = s]$$

Mas a partir de $t+1$, o estado é $S_{t+1}$, e o retorno a partir dali é exatamente $G_{t+1}$. Então:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_t = s]$$

Agora usamos a **lei da esperança total,** condicionando em $S_{t+1} = s'$ e em $A_t = a, e\ obtemos$ **Equação de Bellman para** $V^{\pi}$ (forma explícita):

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s', r \mid s, a)[R(s, a, s') + \gamma V^{\pi}(s')] \tag{14}$$

***Equação de Bellman para*** $\boldsymbol{V^{\pi}(s)}$ ***Vetorizada***

Para obtenção da **forma vetorizada** da **Equação de Bellman para** $V^{\pi}$**,** expandimos a equação (14), como se segue:

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi\,(a \mid s) \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) R(s, a, s') + \gamma \sum_{a \in \mathcal{A}} \pi\,(a \mid s) \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) V^{\pi}(s')$$

O primeiro termo representa a recompensa imediata esperada no estado $s$ ao seguir a política $\pi$.

Definimos $R^{\pi}(s)$ como:

$$R^{\pi}(s) \doteq \sum_{a \in \mathcal{A}} \pi\,(a \mid s) \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) R(s, a, s') \tag{14.a}$$

- Para cada estado $s_i \in \mathcal{S}$, $R^{\pi}(s_i)$ gera um valor escalar.
- Agrupando todos os estados em um vetor de dimensão $|\mathcal{S}| \times 1$:

$$R^{\pi} = \begin{bmatrix} R^{\pi}(s_1) \\ R^{\pi}(s_2) \\ \vdots \\ R^{\pi}(s_{|\mathcal{S}|}) \end{bmatrix}$$

No segundo termo, observe a parte que multiplica $V^{\pi}(s')$:

$$\gamma \sum_{s' \in \mathcal{S}} \left( \sum_{a \in \mathcal{A}} \pi\,(a \mid s) P(s' \mid s, a) \right) V^{\pi}(s')$$

Definimos a probabilidade de transição direta do estado $s$ para $s'$ sob a política $\pi$ como:

$$P^{\pi}(s' \mid s) \doteq \sum_{a \in \mathcal{A}} \pi\,(a \mid s) P(s' \mid s, a) \tag{14.b}$$

Esta é a entrada da linha $s$ e coluna $s'$ na matriz de transição $P^{\pi}$.

Para um ambiente com $N = |\mathcal{S}|$ estados, $P^{\pi}$ é uma matriz estocástica de dimensão $N \times N$:

$$P^{\pi} = \begin{bmatrix} P^{\pi}(s_1 \mid s_1) & P^{\pi}(s_2 \mid s_1) & \dots & P^{\pi}(s_N \mid s_1) \\ P^{\pi}(s_1 \mid s_2) & P^{\pi}(s_2 \mid s_2) & \dots & P^{\pi}(s_N \mid s_2) \\ \vdots & \vdots & \ddots & \vdots \\ P^{\pi}(s_1 \mid s_N) & P^{\pi}(s_2 \mid s_N) & \dots & P^{\pi}(s_N \mid s_N) \end{bmatrix}$$

Substituindo $R^{\pi}(s)$ e $P^{\pi}(s' \mid s)$ de volta na equação escalar:

$$V^{\pi}(s) = R^{\pi}(s) + \gamma \sum_{s' \in \mathcal{S}} P^{\pi}\,(s' \mid s) V^{\pi}(s')$$

Em notação matricial para todos os estados simultaneamente:

$$V^{\pi} = R^{\pi} + \gamma P^{\pi} V^{\pi} \tag{15}$$

Onde:

- $V^{\pi}$ = vetor | $S$ |× 1
- $R^{\pi}$ = vetor | $S$ |× 1 com recompensas esperadas para cada estado (pela equação (14.a))
- $P^{\pi}$ = matriz | $S$ |×| $S$ | de probabilidades de transição (pela equação (14.b))

A equação para $V^{\pi}$ pode ser resolvida de forma recursiva partindo de um valor inicial

$$V_0^{\pi} = \begin{bmatrix} V_0(s_1) \\ V_0(s_2) \\ \vdots \\ V_0(s_N) \end{bmatrix}$$

por ser uma função de ponto fixo utilizando a equação (15) ou por inversão de matrizes, como a seguir**:**

$$V^{\pi} = (I - \gamma P^{\pi})^{-1} R^{\pi} \tag{16}$$

*Onde I é a matriz identidade* $|\mathcal{S}| \times |\mathcal{S}|$. *A matriz* $(I - \gamma P^{\pi})$ *é sempre invertível para* $0 \leq \gamma < 1$*)*

***Condição de contorno da Equação de Bellman para estados terminais***

É comum estabelecer recompensa R igual a zero para permanecer em um **estado terminal**. Dessa forma a equação para o cálculo de $V^{\pi}$ ficando no estado terminal se resume a:

$$V^{\pi} = 0 + \gamma P^{\pi} V^{\pi}$$

Como $\gamma < 1$ então $V^{\pi}$ (terminal) vai tender pra zero - o que pode levar k passos do algoritmo devido ao fator $\gamma^k$ que aparece na recursão - é comum estabelecer $V^{\pi}$ igual a zero como condição de contorno para todos os estados terminais.

***Equação de Bellman para*** $\boldsymbol{Q^{\pi}(s,a)}$

Para obter a função de estado-ação partimos da equação (11):

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}[G_t, \mid S_t = sA_t = a]$$

E fazendo o mesmo tipo de expansão realizado para $V^{\pi}(s)$, obtemos:

$$Q^{\pi}(s,a) = \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) \left[ R(s,a,s') + \gamma \sum_{a' \in \mathcal{A}} \pi\,(a' \mid s') Q^{\pi}(s', a') \right]$$

Substituindo a expressão correspondente por $V^{\pi}$ usando-se a equação (12) chega-se em:

$$Q^{\pi}(s,a) = \sum_{s'} P\,(s', r \mid s, a)[R(s,a,s') + \gamma V^{\pi}(s')] \tag{17}$$

***Equação de Bellman para $Q^{\pi}(s,a)$ na forma vetorizada***

O desenvolvimento da Equação de Bellman na forma vetorizada admite duas abordagens distintas, a depender do tratamento dado ao espaço de dimensões envolvidas $(\mathcal{S}, \mathcal{A}, \mathcal{S}')$:

- **Forma Achatada** $(\mathcal{S} \times \mathcal{A}) \times \mathcal{S}'$**:** As dimensões de estado e ação originais são colapsadas em um espaço linearizado 2D de tamanho $|\mathcal{S}| \cdot |\mathcal{A}|$. Essa transformação reduz o operador de transição a uma matriz bidimensional, o que possibilita a **solução analítica em forma fechada por inversão direta de matrizes** — ainda que restrita a problemas de pequena escala devido ao custo computacional $O((|\mathcal{S}||\mathcal{A}|)^3)$.
- **Forma Não Achatada** $(\mathcal{S}, \mathcal{A}, \mathcal{S}')$**:** Preserva-se a estrutura nativa dos espaços por meio de um tensor de ordem 3. Embora essa formulação impeça a inversão matricial direta, ela permite **aproveitar plenamente o paralelismo de execução em GPUs** via operações de contração tensorial (como a operação de Einstein einsum), tornando-se a escolha ideal para avaliação de políticas por métodos iterativos de ponto fixo.

***Equação de Bellman para $Q^{\pi}(s,a)$ na forma vetorizada achatada***

Substituição de $V^{\pi}(s')$ da equação (12) em $Q^{\pi}(s,a)$ *da equação* (17)**:**

$$Q^{\pi}(s,a) = \sum_{s\prime} P\,(s' \mid s,a)\left[R(s,a,s') + \gamma \sum_{a\prime \in \mathcal{A}} \pi\,(a' \mid s')Q^{\pi}(s',a')\right] \tag{17.a}$$

Expansão dos Termos:

$$Q^{\pi}(s,a) = \underbrace{\sum_{s\prime} P\,(s' \mid s,a)R(s,a,s')}_{\overbrace{R(s,a)}} + \gamma \sum_{s\prime} P\,(s' \mid s,a) \sum_{a\prime \in \mathcal{A}} \pi\,(a' \mid s')Q^{\pi}(s',a')$$

Reescrevendo com o Termo de Recompensa Esperada $R(s,a)$**:**

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s\prime} \sum_{a\prime} P\,(s' \mid s,a)\pi(a' \mid s')Q^{\pi}(s',a') \tag{17.b}$$

$$Q^{\pi}_{flat} = R_{flat} + \gamma P_{flat} Q^{\pi}_{flat} \tag{18}$$

Onde:

- $Q^{\pi}_{flat} = Q^{\pi}$ (s, a) = vetor $|\,S\,| \cdot |\,A\,| \times 1$
- $R_{flat} = R(s,a)$ = vetor $|\,S\,| \cdot |\,A\,| \times 1$ com recompensas médias tal que:

$$R(s,a) = \sum_{s'} P\,(s',\ r \mid s,\ a).R(s,\ a,\ s')$$

- $P_{flat}$ = matriz $|\,S\,| \cdot |\,A\,| \times |\,S\,||\,A\,|$ de probabilidades de transição composta

$$P_{flat}\big((s,a),(s',a')\big) = P(s' \mid s,a) \cdot \pi(a' \mid s')$$

A probabilidade de sair do par $(s,a)$ e ir para o par $(s',a')$ ocorre em dois passos dependentes em cadeia:

- O ambiente transita do estado $s$ para $s'$ dada a ação $a$ com probabilidade $P(s' \mid s,a)$.
- Chegando em $s'$, o agente escolhe a ação $a'$ dada a política com probabilidade $\pi(a' \mid s')$
- Construímos direto uma **única matriz quadrada $P_{flat}$ de dimensão** $(SA \times SA)$:
- $P_{flat} =$

$$\begin{bmatrix} P(s_1'|s_1,a_1)\pi(a_1'|s_1') & P(s_1'|s_1,a_1)\pi(a_2'|s_1') & \dots & P(s_N'|s_1,a_1)\pi(a_M'|s_N') \\ P(s_1'|s_1,a_2)\pi(a_1'|s_1') & P(s_1'|s_1,a_2)\pi(a_2'|s_1') & \dots & P(s_N'|s_1,a_2)\pi(a_M'|s_N') \\ \vdots & \vdots & \ddots & \vdots \\ P(s_1'|s_M,a_M)\pi(a_1'|s_1') & \dots & \dots & P(s_N'|s_M,a_M)\pi(a_M'|s_N') \end{bmatrix}$$

A solução por inversão para o caso vetorial achatado é mostrada a seguir:

$$Q_{flat}^{\pi} = \left(I_{SA} - \gamma P_{flat}\right)^{-1} R_{flat} \tag{19}$$

Para obtenção de $Q^{\pi}$ diretamente de $V^{\pi}$ *em* notação vetorial/tensorial combinamos as equações (14.b) e (17.b):

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s\prime} P\,(s' \mid s,a)\, V^{\pi}(s') \tag{19.a}$$

$$Q_{flat}^{\pi} = R_{flat} + \gamma\, P\, V^{\pi} \tag{20}$$

onde:

- $R_{flat} \in \mathbb{R}^{SA}$ (recompensas achatadas), (S.A, )
- $P \in \mathbb{R}^{SA \times S}$ $(P(s' \mid s,a))$, (S.A, S)
- $V^{\pi} \in \mathbb{R}^{S}$, (S, )
- $Q^{\pi} \in \mathbb{R}^{SA}$ , (S.A, )

O produto $\gamma\, P\, V^{\pi}$ dá um vetor em $\mathbb{R}^{SA}$, com entrada $(s,a)$ equivalente a $\gamma \sum_{s'} P\,(s', \mid sa) V^{\pi}(s')$ e compatível para somar com $R$ e obter o $Q^{\pi}$.

***Equação de Bellman para $Q^{\pi}(s,a)$ na forma vetorizada não achatada***

Partimos as equação (19.a) na qual a operação $\sum_{s'} P\,(s' \mid s,a) V^{\pi}(s')$ representa uma **contração tensorial** entre a 3ª dimensão de $T$ e o vetor $V^{\pi}$, onde:

- **Tensor de Transição ($T$):** Tensor de ordem 3 de dimensão $(S,A,S)$, onde $T_{s,a,s'} = P(s' \mid s,a)$.
- **Matriz de Recompensa Esperada ($R$):** Matriz de dimensão $(S,A)$, onde $R_{s,a} = R(s,a)$.

- **Vetor de Valores de Estado ($V^{\pi}$):** Vetor de dimensão $(S,)$.
- **Matriz de Valores de Ação ($Q^{\pi}$):** Matriz de dimensão $(\mathrm{S}, \mathrm{A})$.

$$Q_{s,a}^{\pi} = R_{s,a} + \gamma . Einsum(T, V_n^{\pi}) \tag{21}$$

A equação (21) pode ser apresentada na forma de Eisntein como a soma ponderada dos produtos entre fatias do Tensor T e os valores individuais $V_n^{\pi}$ para todos os possíveis próximos estados s' que são utilizados como pesos nessa soma. Em notação de convenção de somatório de Einstein ( xp.einsum('san,n->sa', T, V) **):**

$$Q_{s,a}^{\pi} = R_{s,a} + \gamma \sum_{n=1}^{S} T_{s,a,n}\ V_n^{\pi} \tag{22}$$

## 3.2 – Equações de Bellman para obtenção da Política Ótima

***Equação de Bellman Ótima para a função de valor do estado***

O próximo passo será adaptar as equações obtidas anteriormente para obtenção da política ótima. Se eu estou no estado $s$ e quero maximizar meu retorno futuro, devo escolher a ação que dá o maior Q-valor:

$$\text{Melhor ação em } s = \arg\max_{a} Q^{\pi}(s,a)$$

O **valor máximo** que posso obter em $s$ é:

$$\max_{a} Q^{\pi}(s,a)$$

Para uma política **ótima** $\pi^*$, o agente **sempre** escolhe a ação que maximiza $Q^{\pi^*}(s,a)$:

$$\pi^*(a \mid s) = \begin{cases} 1 & \text{se } a = \arg\max_{a} Q^{\pi^*}(s,a) \\ 0 & \text{caso contrario} \end{cases}$$

Ou seja, a política ótima é **determinística** e **gulosa (*greedy*)** em relação a $Q^*$.

Portanto:

$$V^*(s) = \max_{a} Q^*(s,a) \tag{23}$$

Isso difere da relação para uma política genérica estocástica, que é expressa por uma **média ponderada** ($\sum_a \pi\,(a \mid s) Q^{\pi}(s,a)$). No caso da política ótima determinística $V^*(s)$, a média vira um **máximo** porque todo o peso vai para a melhor ação.

> **Obs:** No caso de empate entre mais de uma ação ao aplicar a equação (23) pode-se gerar uma **política estocástica** a sortear de forma equiprovável qual ação que o agente deverá tomar.

Pode-se deduzir a equação de Bellman para política ótima $\pi^*(a/s)$ (Equação da Otimalidade de Bellman) da função de estado-valor $V^*$com os seguintes passos:

**Passo 1:** Partimos da definição de $V^*$:

$$V^*(s) = \max_a Q^*(s,a)$$

**Passo 2:** Usamos a definição de $Q^*$ (que é a mesma para qualquer política pela equação (17)):

$$Q^*(s,a) = \sum_{s'} P\,(s', r \mid s,a)[R(s,a,s') + \gamma V^*(s')]$$

**Passo 3:** Substituímos a equação do passo 2 na equação do passo 1 e obtemos a Equação de Bellman Ótima para a função de estado-valor $V^*(s)$:

$$V^*(s) = \max_a \sum_{s'} P\,(s', r \mid s,a)[R(s,a,s') + \gamma V^*(s')] \tag{24}$$

Assim temos que:

$$V^*(s) = V^{\pi *}(s) \geq V^{\pi}(s)$$

A função estado-ação para um política ótima é maior que outra função estado-ação para qualquer outra política em qualquer estado.

***Equação de Bellman Ótima para a função de valor de estado-ação***

De forma análoga pode-se deduzir a equação de Bellman para política ótima (Equação da Otimalidade de Bellman) da função de ação-valor $Q^*(s,a)$ com os seguintes passos:

**Passo 1:** Partimos da definição de $Q^*$:

$$Q^*(s,a) = \sum_{s'} P\,(s', r \mid s,a)[R(s,a,s') + \gamma V^*(s')]$$

**Passo 2:** Usamos a relação $V^*(s') = \max_{a'} Q^*(s',a')$ e obtemos a função de ação-valor $Q^*(s,a)$:

$$Q^*(s,a) = \sum_{s'} P\,(s', r \mid s,a)\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right] \tag{25}$$

As equações para $V^*(s)$ e $Q^*(s,a)$ são do tipo ponto fixo (x = f(x)) e podem ser resolvidas a partir de um valor inicial $x_0$ de forma recursiva.

## 3.3 – Métodos de Solução por Iteração de Valor, Iteração de Política e GPI

Em sequencia são apresentados três métodos de solução para a obtenção da Política Ótima $\pi^*(a \mid s)$ **: Iteração de Valor, Iteração de Política e GPI** (*Generalized Policy Iteration*).

***Método de Iteração de Valor***

O método de solução de política ótima $\pi^*(a \mid s)$ por **iteração de valor** parte de um valor inicial $V_0(s)$ para todo estado s. A equação (24) para $V^*(s)$ é então resolvida de forma recursiva até a convergência. Em seguida utiliza-se $V^*(s)$ no cálculo de $Q^{\pi^*}(s,a)$ pela equação (17) para obter a política ótima através da operação $a = \arg\max_a Q^{\pi^*}(s,a)$.

O algoritmo em pseudocódigo da solução por **iteração por valor** é apresentado na Figura 5 conforme extraído da referência [ 1].

**Value Iteration, for estimating $\pi \approx \pi_*$**

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$

**Figura 5** – Algoritmo por Iteração de Valor (extraído da referência [1])

***Método de Iteração de Política***

O método de solução de política ótima $\pi^*(a \mid s)$ por **iteração de política** parte de uma política arbitrária inicial $\pi(a \mid s)$. Avalia-se $V^\pi(s)$ recursivamente com a equação (14) para essa política até obter a convergência. Observe que não deve-se utilizar max nessa avaliação pois não se está procurando $V_*(s)$.

Em seguida utiliza-se $V^\pi(s)$ no cálculo de $Q^\pi(s,a)$ pela equação (17) para **melhorar** a política através da operação $a = \arg\max_a Q^\pi(s,a)$, obtendo uma nova política $\pi_{novo}$. Se $\pi_{novo} \neq \pi$, repete-se o processo com a nova política. Quando $\pi_{novo} = \pi$, então $\pi = \pi^*$.

O algoritmo em pseudocódigo da solução por **iteração de política** é apresentado na Figura 6 conforme extraído da referência [ 1].

**Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$**

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
      $\Delta \leftarrow 0$
      Loop for each $s \in \mathcal{S}$:
         $v \leftarrow V(s)$
         $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))\left[r + \gamma V(s')\right]$
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   *policy-stable* $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
      *old-action* $\leftarrow \pi(s)$
      $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$
      If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

**Figura 6** – Algoritmo para Iteração de Política (extraído da referência [1])

A **avaliação de política determinística**, no passo 2 do algoritmo de Iteração de Política, trata da obtenção de $V\pi(s)$ a partir da equação (14), a função de estado para uma dada política $\pi \ (a \mid s)\ determinística$, ou seja:

$$V^{\pi}(s) = \sum_{s'} P\left(s', a \mid s, \pi(s)\right)[R(s, \pi(s), s') + \gamma V^{\pi}(s')]$$

A **avaliação de política para política estocástica**, no passo 2 do algoritmo de Iteração de Política, trata da avaliação de $V\pi(s)$, a função de estado-ação para uma dada política $\pi \ (a \mid s)\ estocástica$, que pode ser apresentada na forma escalar ou vetorial, como a seguir:

$$V^{\pi}(s) = \sum_{a} \pi\,(a \mid s) \sum_{s'} P\,(s', a \mid s, a)[R(s, a, s') + \gamma V^{\pi}(s')]$$

> **Obs:** Utilizada na avaliação de políticas estocásticas (ex: $\epsilon$-greedy).

***Generalized Policy Iteration***

O ***Generalized Policy Iteration*** **(GPI)** é um conceito que se baseia na generalização do processo de iteração por política que busca **economizar tempo computacional** ao não exigir a convergência completa da avaliação da função valor $V(s)$ antes de realizar a melhoria da política. Em vez de resolver exatamente o sistema de equações de Bellman para $V^{\pi}$ até a

convergência (como no Policy Iteration clássico), o GPI intercala passos de avaliação incompleta de $V(s)$ e melhoria da política, permitindo que $V(s)$ seja atualizada apenas algumas vezes (ou até uma única vez) antes de se melhorar a política em cada passo.

### *Modified Policy Iteration*

O **MPI (Modified Policy Iteration)** proposto por Puterman & Shin (1978) é um algoritmo GPI que, em cada iteração, aplica **k vezes o operador de Bellman da política atual** para realizar uma avaliação aproximada do valor, e em seguida realiza a melhoria da política. Ele unifica **Value Iteration** (k=1) e **Policy Iteration** (k=∞) como dois casos extremos.

```
Modified Policy Iteration (GPI)
(using partial iterative policy evaluation)

1. Initialization

   Initialize V(s) ∈ R and π(s) ∈ A(s) arbitrarily
   for all s ∈ S

2. Generalized Policy Iteration

   Repeat:

      2.1 Partial Policy Evaluation

          Perform a limited number k of iterative policy
          evaluation sweeps, without requiring convergence.

          For each sweep:

              For each s ∈ S:

                  V(s) ← Σs',r p(s', r | s, π(s)) [r + γV(s')]

      2.2 Policy Improvement

          policy-stable ← true

          For each s ∈ S:

              old-action ← π(s)

              π(s) ← argmax a Σs',r p(s', r | s, a) [r + γV(s')]

              If old-action ≠ π(s), then
                  policy-stable ← false

          If policy-stable, then stop and return
              V ≈ V* and π ≈ π*

   until policy-stable
```

**Figura 7** – Pseudocódigo para Generalized Policy Iteration

## 3.4 – Comparação entre os Métodos de Solução

A tabela a seguir apresenta uma comparação da ordem de complexidade do custo computacional dos algoritmos estudados para solução do MDP:

**Tabela 1** – Comparação do Custo Computacional dos Algoritmos MDP

| Método | Custo por iteração | Nº de iterações (k) | Custo Total (assintótico) |
|---|---|---|---|
| ***Policy Iteration*** **(PI)** | $O(\|A\|\|S\|^2 + \|S\|^3)$ | Muito baixo (tipicamente quase independente de $\|S\|$) | Domina para $\|S\|$ grande — o termo $\|S\|^3$ vira o gargalo |
| ***Value Iteration*** **(VI)** | $O(\|A\|\|S\|^2)$ | Alto, cresce com $1/(1-\gamma)$ | Sem termo cúbico, mas nº de iterações explode quando $\gamma \to 1$ |
| ***Modified Policy Iteration*** **MPI)** | $O(k \cdot \|A\|\|S\|^2)$ | Decresce conforme k aumenta, aproximando-se do de PI | Sem termo cúbico e poucas iterações — melhor trade-off para $\|S\|$ grande |

O valor k é considerado um hiperparâmetro a ser ajustado para o algoritmo de MPI, e pode fazer o MPI se aproximar mais de VI ou de PI conforme o seu valor:

- k pequeno demais: se aproxima de VI, herdando o problema de muitas iterações externas quando $\gamma$ é próximo de 1; e
- k grande demais: embora não possua o termo cúbico, o custo por iteração $k \cdot |A||S|^2$ pode acabar comparável ao de PI se k crescer demais, e você perde a vantagem.

MPI possui desempenho computacional melhor que PI e VI para MDPs com muitos estados, desde que k seja ajustado ao problema — ele é o único dos três algoritmos que evita simultaneamente o termo cúbico de PI e a explosão de iterações de VI quando $\gamma$ é alto.

As soluções por inversão de matrizes foram apresentadas apenas como base teórica e não foram utilizadas nas simulações em Python deste artigo. O motivo principal é do ponto de vista numérico, a matriz $I - \gamma P^{\pi}$ torna-se progressivamente mal-condicionada à medida que $\gamma \to 1$, de modo que sua inversão amplifica erros de arredondamento — problema que a iteração de ponto fixo não apresenta, pois seu erro decai geometricamente com fator $\gamma$, independentemente do condicionamento.

Todos os algoritmos que vimos nesta seção dependem da avaliação da função valor $V(s)$ em todos os estados em cada passo e por isso são classificados com **algoritmos síncronos**. Os mesmos algoritmos podem ser implementados na versão assíncrona permitindo propagar a avaliação de V(s) em cada estado individualmente. Porém os **algoritmos assíncronos** não são apropriados para vetorização e uso de GPUs.

# 4. Métricas para Avaliação de Soluções

As seguintes métricas são podem ser utilizadas para avaliação das soluções: número de passos para convergência e tempo de execução. Neste estudo optou-se por usar o tempo de execução.

# 5. Exemplos de aplicação de MDP e Resultados:

Esta seção aborda exemplos de: avaliação de função de estado a partir de uma política conhecida para o exemplo 3.5 do livro [1] [Sutton & Barto], obtenção da política ótima para jogo de tabuleiro do robô logístico apresentado na Figura 3, obtenção da política ótima para jogo de tabuleiro do exemplo 3.8 do livro [1] [Sutton & Barto], obtenção da política ótima para o jogo **FrozenLake** disponível no pacote Gymnasium, obtenção de política ótima para o jogo de tabuleiro do Wumpus do livro [7]. Os códigos em Python utilizados no exemplos estão disponibilizados em [11].

As seguintes convenções foram utilizadas na implementação em Python dos exemplos de jogos de tabuleiro:

- $S_0, S_1, S_2, \ldots$ índices dos estados crescem de cima pra baixo e da esquerda para direita do tabuleiro; e
- Posição (x, y) de origem no tabuleiro corresponde ao canto inferior esquerdo do tabuleiro – posição (0,0).

Os notebooks da implementação em Python dos exemplos apresentados a seguir encontram-se em [11].

## 5.1 – Avaliação de função de estado $V^{\pi}(s)$ a partir de uma política conhecida$\pi$ $(a \mid s)$

O Exemplo 3.5 do livro [1] [Sutton & Barto] trata de um tabuleiro 5 x 5 com as seguintes regras para construção da Dinâmica do MDP:

- Cada posição no tabuleiro está associada a um estado;
- Ações de movimentação em cada estado/posição: pra cima, direita, pra baixo e esquerda;
- Recompensa de +10 por qualquer ação no estado $S_1$ fazendo a transição para o estado $S_{21}$;
- Recompensa de +5 por qualquer ação no estado $S_3$ fazendo a transição para o estado $S_{13}$;
- Todos os outros movimentos nos demais estados têm recompensa zero;
- Considere dada uma política equiprovável em cada um dos estados, isto é, a probabilidade de tomar qualquer das ações em um estado é igual a ¼.

A Figura 8 apresenta a equação e o fluxo de dados para a avaliação de política. O criador de política (*Policy Builder*) gera uma matriz 5 x 4 com todas as linhas iguais a ¼. O gerador de Dinâmica do Tabuleiro (*GridDynBuilder*) produz um dicionário de dinâmica na estrutura da Figura 4 a partir das regras definidas para o tabuleiro. Utilizando a matriz de política e o dicionário da dinâmica o Avaliador de Política calcula $V^{\pi}(s)$ recursivamente com a equação (14) partindo do valor igual a zero e a constante gamma igual a 0,9. O Visualizador de

Resultados plota o desenho do Tabuleiro indicando a política com setas e o valor da função de estado em cada posição.

A Figura 9 mostra a arquitetura do projeto de classes em Python para Avaliação de Política.

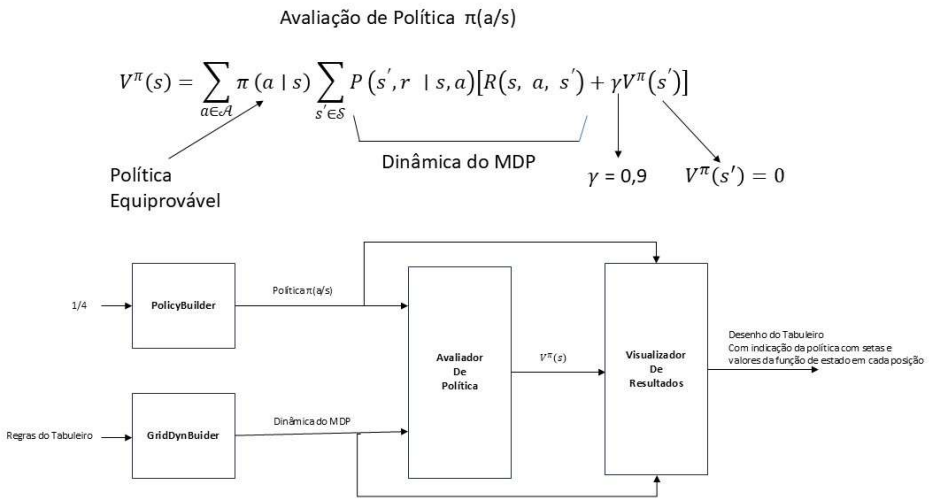



**Figura 8** – Equação e fluxo de dados para Avaliação de Política

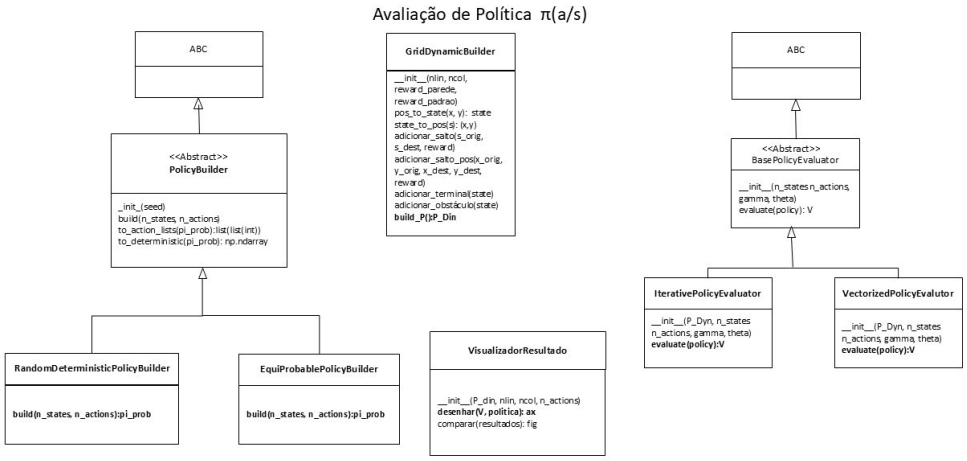



**Figura 9** – Arquitetura para Avaliação de Política

A Tabela 2 descreve os padrões de projetos utilizados na arquitetura de classes para avaliação de política.

**Tabela 2** – Padrões de Projeto para Avaliação de Política

| Classe | Padrão de Projeto | Função |
|---|---|---|
| PolicyBuilder (e suas subclasses RandomDeterministicPolicyBuilder, EquiProbablePolicyBuilder) | Factory Method (Pg. 107 de [8]) | Define uma interface para criar um objeto (build), mas deixa as subclasses decidirem qual classe instanciar. Isso permite que a criação da política (pi_prob) varie independentemente do cliente que a utiliza. A classe ABC no topo sugere uma hierarquia de criação. |
| | Template Method (Pg. 325 de [8]) | Define o esqueleto de um algoritmo na superclasse (build), mas permite que as subclasses (Random... e EquiProbable...) redefinam certos passos específicos do algoritmo sem alterar sua estrutura geral. |
| BasePolicyEvaluator (e suas subclasses IterativePolicyEvaluator, VectorizedPolicyEvaluator) | Strategy (Pg. 315 de [8]) | Define uma família de algoritmos (avaliação iterativa vs. vetorizada), encapsula cada um deles e os torna intercambiáveis. O BasePolicyEvaluator define a interface comum (evaluate), permitindo que o algoritmo de avaliação varie independentemente do contexto que o usa. |
| GridDynamicBuilder | Builder (Pg. 97 de [8]) | Separa a construção de um objeto complexo (P_Din) da sua representação, permitindo que o mesmo processo de construção crie diferentes representações. A classe fornece métodos passo-a-passo (adicionar_salto, adicionar_terminal, etc.) para construir o ambiente de grid complexo antes de retornar o objeto final. |

| Classe | Padrão de Projeto | Função |
|---|---|---|
| VisualizadorResultado | Façade<br>(Pg. 185 de [8]) | Fornece uma interface unificada para um conjunto de interfaces em um subsistema. O visualizador esconde a complexidade de interagir com bibliotecas de plotagem (provavelmente Matplotlib) e estruturas de dados internas, oferecendo métodos simples como desenhar e comparar para o cliente. |

A Figura 10 apresenta o resultado para o tabuleiro do Exemplo 3.5 do [Sutton & Barto] [1] validando a implementação por coincidir com o resultado mostrado neste livro. O gerador de Dinâmica do Tabuleiro (*GridDynBuilder*) gera um dicionário de dinâmica na estrutura da Figura 4 a partir das regras definidas para o tabuleiro.

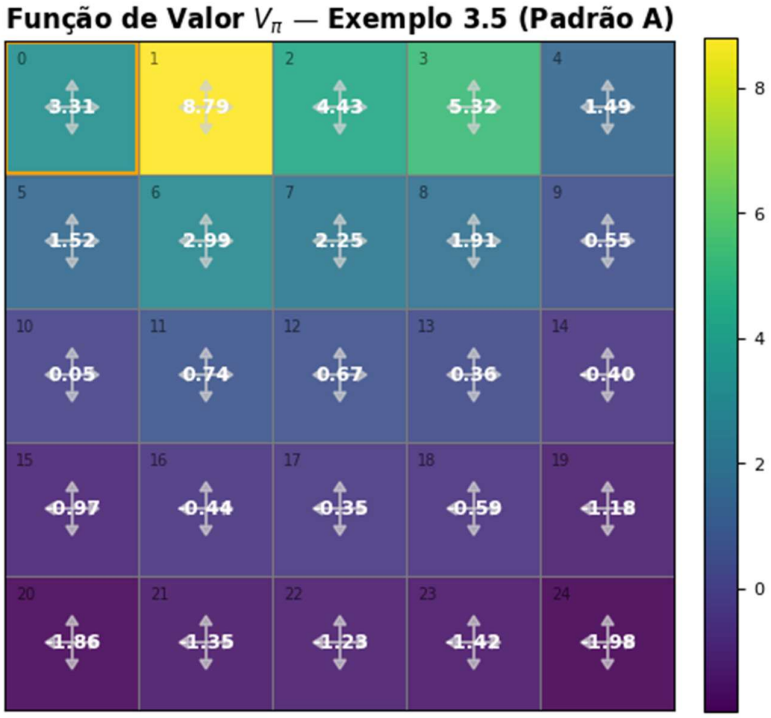



**Figura 10** – Avaliação de Política do Exemplo 3.5 de [Sutton&Barto] [1]

> **Obs:** Opção A - Estado $S_0$ no canto superior esquerdo (posição (0, 4) em (x , y)).

## 5.2 – Obtenção da Política Ótima $\pi^*(a \mid s)$ conhecida a Dinâmica de jogos de Tabuleiro

A Figura 11 apresenta as equações e o Fluxo de Dados empregados nos algoritmos de VI, PI e MPI para obtenção da Política Ótima para jogos de Tabuleiro a partir da sua dinâmica. O

gerador de Dinâmica do Tabuleiro (*GridDynBuilder*) produz um dicionário de dinâmica na estrutura da Figura 4 a partir das regras definidas para o tabuleiro. A classe Tabuleiro gera um ambiente com interface padrão do pacote *Gymnasium* do Python. O Tabuleiro é processado pelo algoritmo de planejamento a escolher – PI, VI ou MPI – para obtenção da Política Ótima $\pi^*(a \mid s)$. O Visualizador de Resultados plota o desenho do Tabuleiro indicando a política com setas e o valor da função de estado em cada posição.

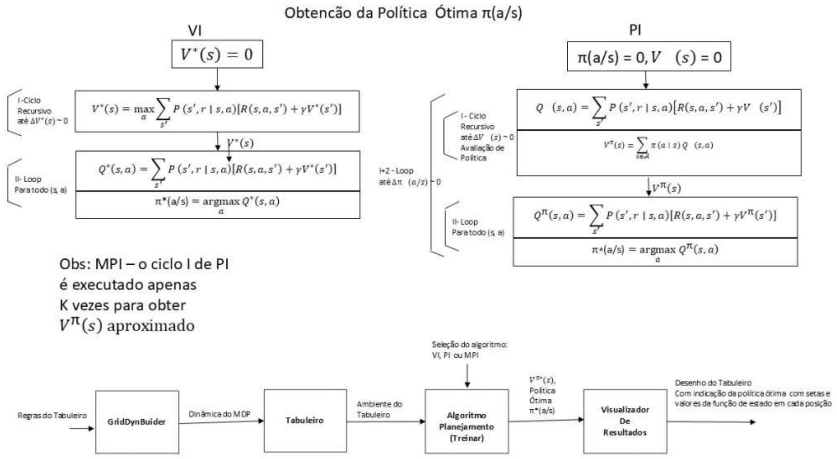



**Figura 11** – Equações e Fluxo de Dados para Obtenção da Política Ótima $\boldsymbol{\pi^*(a \mid s)}$

> ***Nota:*** **Obs:** A formulação matemática apresentada para o ciclo I de PI utiliza a notação geral para políticas estocásticas ($\boldsymbol{\pi(a \mid s)}$). A implementação computacional (Ciclo I) otimiza esta etapa utilizando uma amostra determinística da política, uma vez que o resultado de $\boldsymbol{V^{\pi}(s)}$é matematicamente equivalente para qualquer distribuição de probabilidade entre ações de mesmo valor $\boldsymbol{Q(s,a)}$ótimo.

A Figura 12 mostra a arquitetura do projeto de classes em Python para Obtenção da Política Ótima $\boldsymbol{\pi(a \mid s)}$ para um jogo de Tabuleiro de regras conhecidas.

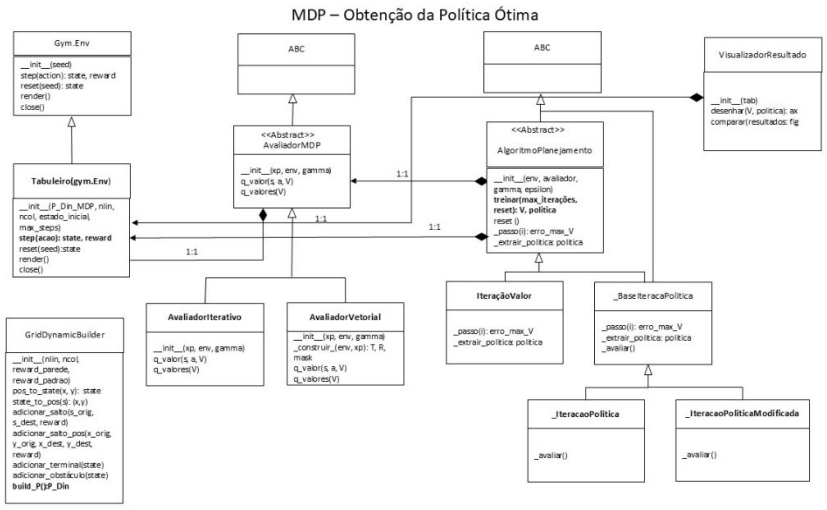



**Figura 12** – Arquitetura para Obtenção de Política Ótima

A Tabela 3 descreve os padrões de projetos utilizados na arquitetura de classes para obtenção da política ótima.

**Tabela 3** – Padrões de Projeto para Obtenção da Política Ótima

| Classe | Padrão de Projeto | Função |
|---|---|---|
| AvaliadorMDP | Strategy<br>(Pg. 315 de [8]) | Interface comum para os algoritmos de avaliação (concreto e vetorial). Permite que o AlgoritmoDePlanejamento trate diferentes métodos de avaliação de forma intercambiável. |
| AvaliadorVetorial | Strategy (Concrete Strategy)<br>(Pg. 315 de [8]) | Implementação concreta da avaliação de políticas utilizando álgebra vetorial/matricial. É uma das estratégias que podem ser usadas pelo planejador. |
| AvaliadorIterativo | Strategy (Concrete Strategy)<br>(Pg. 315 de [8]) | Implementação concreta da avaliação de políticas de forma iterativa. É outra estratégia intercambiável para o AlgoritmoDePlanejamento. |
| AlgoritmoDePlanejamento | Strategy (Context)<br>(Pg. 315 de [8]) | Classe abstrata que define o contrato para os algoritmos de planejamento. Atua como o **Contexto** no padrão Strategy, utilizando a interface AvaliadorMDP. |
| IteracaoValor | Strategy (Concrete Strategy)<br>(Pg. 315 de [8]) | Implementa o algoritmo de Iteração de Valor. É uma estratégia concreta de planejamento que herda de AlgoritmoDePlanejamento. |

| Classe | Padrão de Projeto | Função |
|---|---|---|
| BaseIteracaoPolitica | Template Method<br>(Pg. 325 de [8]) | Classe abstrata que define o esqueleto do algoritmo de iteração de política. |
| IteracaoPolitica | Template Method<br>(Concrete Class)<br>(Pg. 325 de [8]) | Implementação concreta do algoritmo de iteração de política, herdando de BaseIteracaoPolitica. |
| IteracaoPoliticaModificada | Template Method<br>(Concrete Class)<br>(Pg. 325 de [8]) | Variante do algoritmo de iteração de política, herdando de BaseIteracaoPolitica. |
| Tabuleiro(gym.Env) | Adapter<br>(Pg. 138 de [8]) | Adapta a interface do ambiente Gym.Env para a interface esperada pelo MDP. |
| GridDynamicBuilder | Builder<br>(Pg. 97 de [8]) | Separa a construção de um objeto complexo (P_Din) da sua representação, permitindo que o mesmo processo de construção crie diferentes representações. A classe fornece métodos passo-a-passo (adicionar_salto, adicionar_terminal, etc.) para construir o ambiente de grid complexo antes de retornar o objeto final. |
| VisualizadorResultado | Façade<br>(Pg. 185 de [8]) | Fornece uma interface unificada para um conjunto de interfaces em um subsistema. O visualizador esconde a complexidade de interagir com bibliotecas de plotagem (provavelmente Matplotlib) e estruturas de dados internas, oferecendo métodos simples como desenhar e comparar para o cliente. |

### *5.2.1 – Resultados para o exemplo do Robô Logístico*

A Figura 13 apresenta os resultados para o Tabuleiro 2 x 2 descrito na Figura 3 (Robô Logístico) para os 3 tipos de algoritmos: VI, PI e MPI. Os valores de $V^*(s)$ são mostrados nas células do tabuleiro 2 x 2. As setas vermelhas indicam a Política Ótima $\pi^*(a \mid s)$ que neste caso é determinística ($S_0 \rightarrow S_1 \rightarrow S_3$).

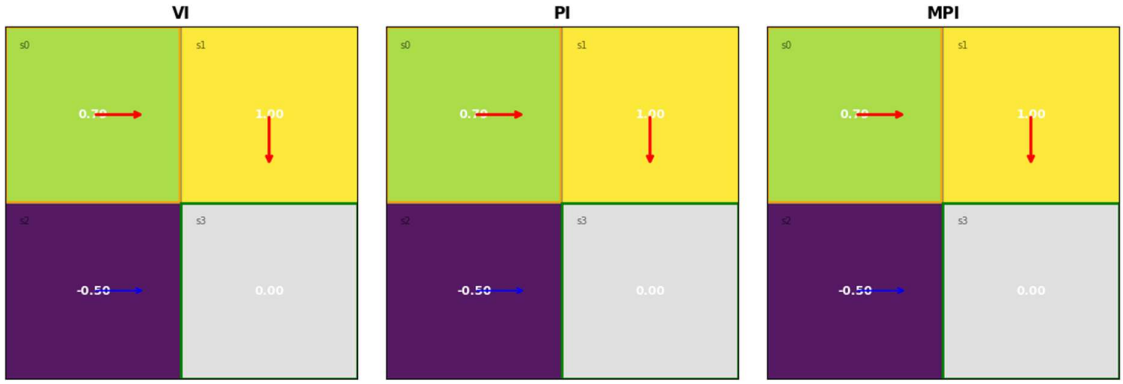



**Figura 13** – Resultados para o exemplo do Robô Logístico

Foram medidos os tempos da execução repetida por 1000 vezes para cada algoritmo – VI, PI e MPI com k=1, para fim de comparação e os resultados são apresentados na Tabela 4 (alternativas iterativa e vetorizada, numpy, cpu do Google Colab).

**Tabela 4** – Tempo de execução para 1000 vezes

| Tempo de Execução (1000 vezes) | VI | PI | MPI com k =1 |
|---|---|---|---|
| Iterativo | 0,3654 segs | 51,4283 segs | 0,5929 segs |
| Vetorizada | 0,2102 segs | 28,6350 segs | 0,3257 segs |

A política ótima obtida está coerente com a Figura 3, pois o caminho alternativo $S_0 \rightarrow S_2 \rightarrow S_3$apresenta maiores recompensas médias negativas. Da mesma forma, os tempos obtidos na Tabela 4 estão compatíveis com as complexidades de computação da Tabela 1. PI é muito mais pesado computacionalmente. O MPI com valor de k ajustado como hiperparâmetro é interessante para ser explorado no caso de uma maior número de estados.

### *5.2.2 – Resultados para o Exemplo 3.8 do livro [Sutton & Barto] [1]*

Vamos abordar agora o mesmo problema do item 5.1 porém ao invés de avaliar uma política conhecida vamos obter a política ótima para o mesmo tabuleiro 5 x 5 com as regras de jogo já descritas anteriormente.

A Figura 14 apresenta os resultados para o Tabuleiro 5 x 5 descrito no exemplo 3.8 do [Sutton & Barto] [1] para os 3 tipos de algoritmos: VI, PI e MPI com k =1. Os valores de $V^*(s)$ são mostrados nas células do tabuleiro 5 x 5. As setas indicam a Política Ótima $\pi^*(a \mid s)$ que neste caso é estocástica. Foram obtidos os mesmos resultados do livro validando a implementação em Python.

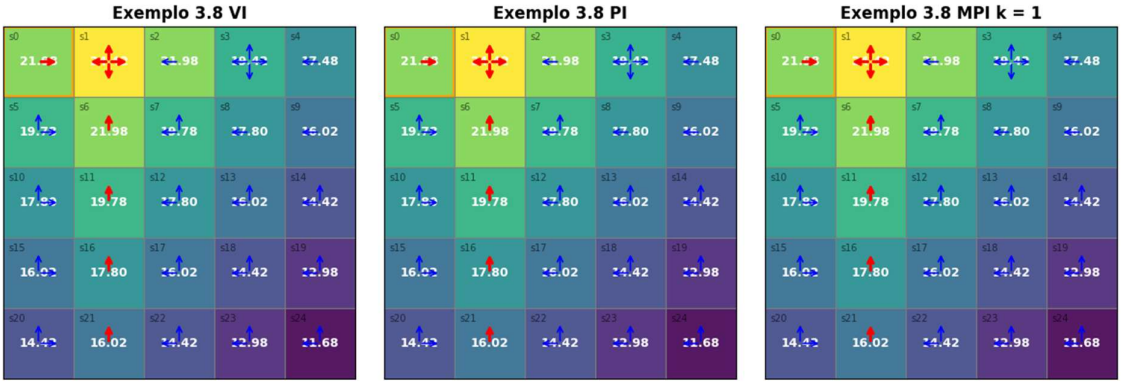



**Figura 14** – Resultados para o Exemplo 3.8 de [Sutton & Barto] [1]

### *5.2.3 – Resultados para o Lago Congelado (Frozen Lake)*

As Figuras 11 e 12 também se aplicam a obtenção da Política Ótima para o jogo de Tabuleiro do Lago Congelado (*Frozen Lake*) do pacote Gymnasium do Python. Porém neste caso a Dinâmica é obtida diretamente do ambiente do Frozen Lake não sendo necessário construir a dinâmica a partir de regras. A dinâmica obtida do Frozen Lake precisa ser convertida para o padrão de codificação das ações utilizado na Classe Tabuleiro como mostrado na Tabela 5. Pode-se escolher a opção pista escorregadia ou não no ambiente da Frozen Lake.

**Tabela 5** – Conversão de códigos de ação

| Ação | Padrão Frozen Lake | Padrão Tabuleiro |
|---|---|---|
| Esquerda | 0 | 3 |
| Pra Baixo | 1 | 2 |
| Direita | 2 | 1 |
| Pra Cima | 3 | 0 |

A Figura 15 apresenta uma possibilidade de cenário sorteada para o jogo do Frozen Lake. O jogo começa no canto superior esquerdo. Há quatro estados terminais de perda (cair no lago) e um estado terminal de sucesso (pegar o presente).



**Figura 15** – Cenário Inicial do Jogo Frozen Lake

A Figura 16 mostra o Fluxo de dados para construção do Tabuleiro Lago Congelado que é posteriormente processado pelo Algoritmo de Planejamento para obtenção da Política Ótima.

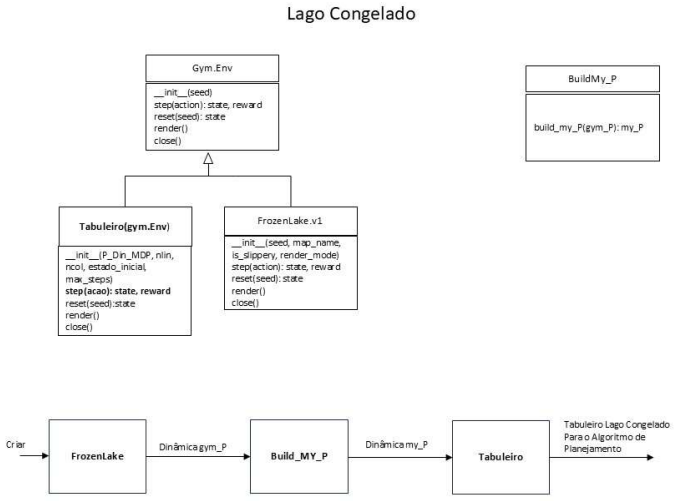



**Figura 16** – Fluxo de Dados para o jogo do Lago Congelado

A Figura 17 apresenta os resultados para o Tabuleiro do Lago Congelado para os 2 tipos de opção: escorregadio ou não. Os valores de $V^*(s)$ são mostrados nas células do tabuleiro 4 x 4. As setas indicam a Política Ótima $\pi^*(a \mid s)$ que neste caso é estocástica.

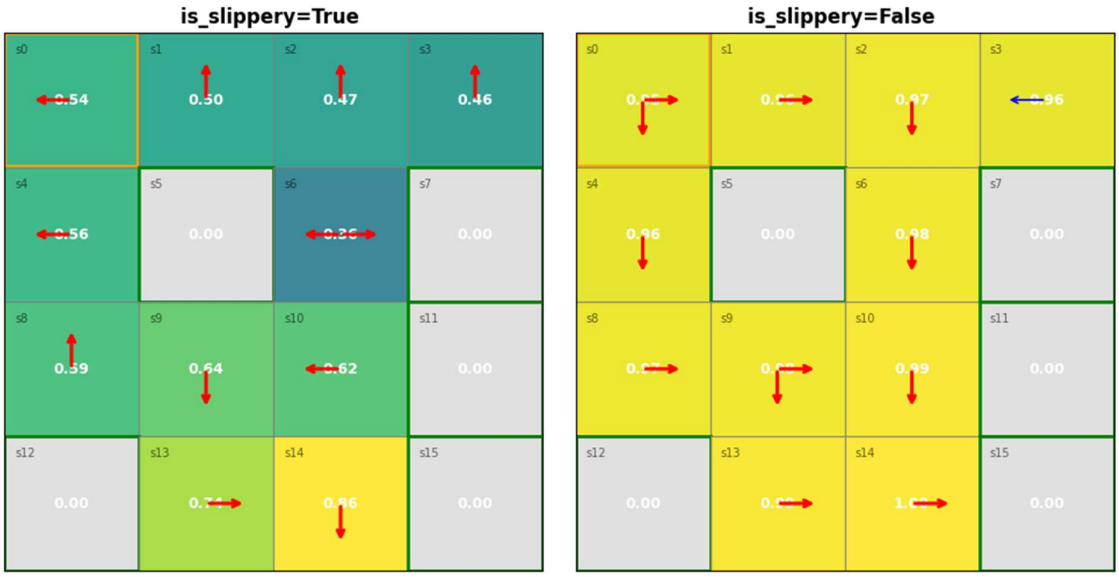



**Figura 17** – Resultados para o Tabuleiro do Lago Congelado

> **Obs:** Nota-se que no caso escorregadio a direção das setas não indica a direção real que o jogador deve seguir.

### *5.2.4 – Resultados para o Jogo do Wumpus Simplificado*

As Figuras 11 e 12 também se aplicam a obtenção da Política Ótima para o jogo de Tabuleiro Wumpus apresentado no capítulo 7 do livro de [Russel, Norvig] [7]. Porém, como mostrado no fluxo de dados da Figura 18, a classe WumpusMDP utiliza a classe GridDynamicBuilder internamente para gerar a dinâmica do Wumpus a partir das regras do jogo. Após a geração dessa dinâmica pode-se criar um tabuleiro Wumpus para que seja processado pelo Algoritmo de Planejamento para obtenção da Política Ótima.

O jogo do Tabuleiro Wumpus simplificado possui as seguintes regras:

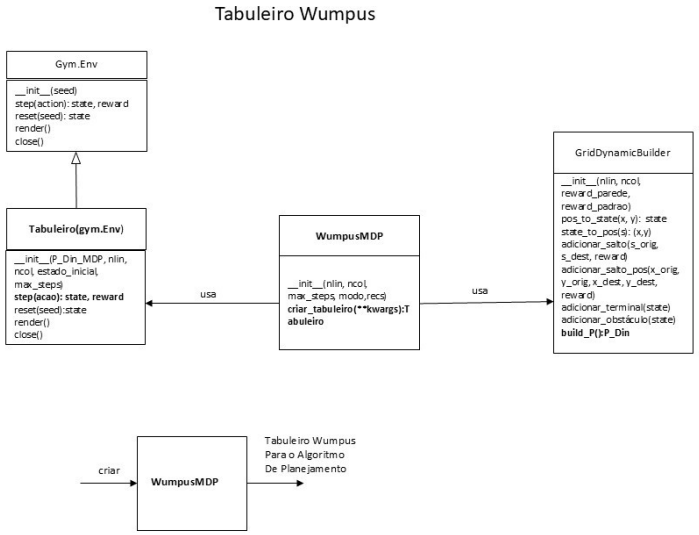



**Figura 18** – Fluxo de Dados do Tabuleiro Wumpus

- Tamanho do tabuleiro n x n (n = 10);
- Número de estados terminais buracos 15% ( 15 para n=10);
- Um único jogador com estado inicial no canto superior esquerdo;
- Um único monstro Wumpus estático;
- Um estado terminal tesouro;
- Vento nas células vizinhas não diagonais aos buracos;
- Cheiro nas células vizinhas não diagonais ao monstro Wumpus;
- Brilho de luz nas células vizinhas não diagonais a tesouro;
- Posições do Wumps, tesouro e buracos sorteadas inicialmente e mutuamente excludentes entre si e o jogador; e
- Possibilidade de superposição dos efeitos de vento, cheiro e luz entre si e com as posições do jogador, do Wumpus, dos buracos e do tesouro.

Objetivos do Jogo:

- Etapa 1 – O jogador move-se até a posição do Tesouro para pegá-lo;
- Etapa 2 – O jogador retorna com o Tesouro até a sua posição inicial.

A Tabela 6 mostra o modelo de recompensas para o jogo do Wumpus.

**Tabela 6** – Modelo de Recompensas para o Jogo do Wumpus

| Tipo de Recompensa (Indo para o estado) | Valor da Recompensa |
|---|---|
| Padrão | -1,0 |

| Tipo de Recompensa (Indo para o estado) | Valor da Recompensa |
|---|---|
| Parede | -2,0 |
| Ouro | +1.000,00 |
| Wumpus | -1.000,00 |
| Buraco | -3.0 |
| Vento | -3.0 |
| Luz | +1.0 |

> **Obs:** Para a Etapa 2 a posição do tesouro torna-se a posição inicial e a posição inicial original torna-se a posição de destino com recompensa igual a +1.000,00 em sua transição para este estado terminal.

A Figura 19 mostra as posições resultantes do sorteio no início da Etapa 1.

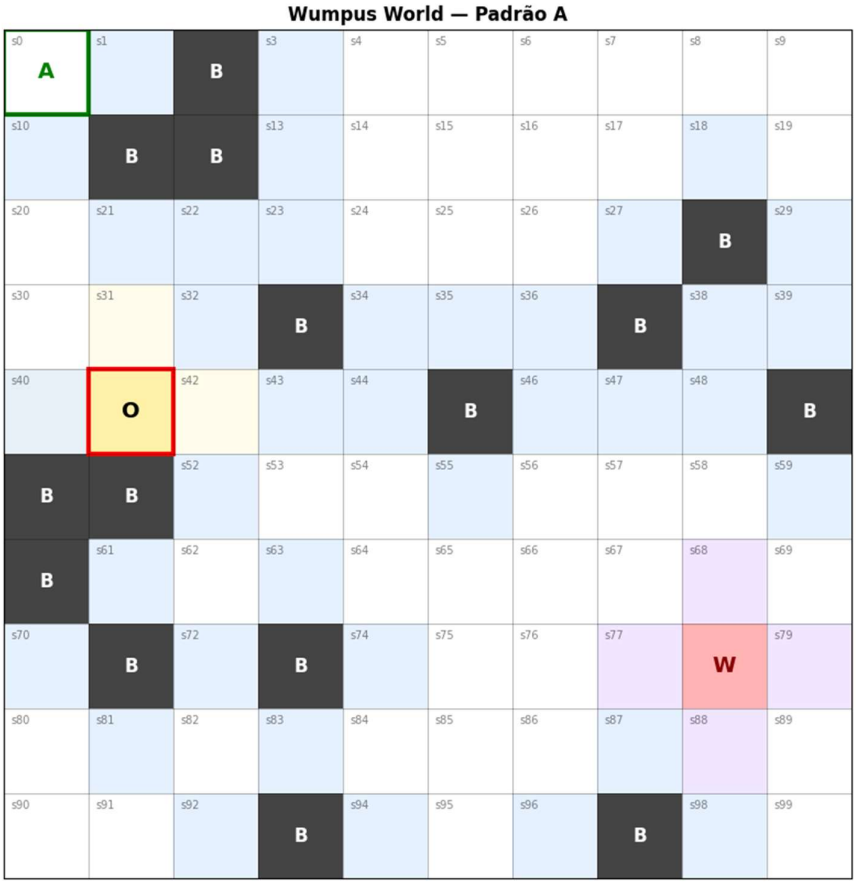



**Figura 19** – Posições Sorteados no início da Etapa 1

> **Obs:** A – Posição Inicial; B- Buraco; O- Tesouro; W – Wumpus; efeitos de vento, cheiro e luz em células vizinhas com diferentes cores.

A Figura 20 apresenta os resultados para a Etapa 1 do jogo do Wumpus. Os valores de $V^*(s)$ são mostrados nas células do tabuleiro 10 x 10. As setas em vermelha indicam a Política Ótima $\pi^*(a \mid s)$ a partir do estado inicial.

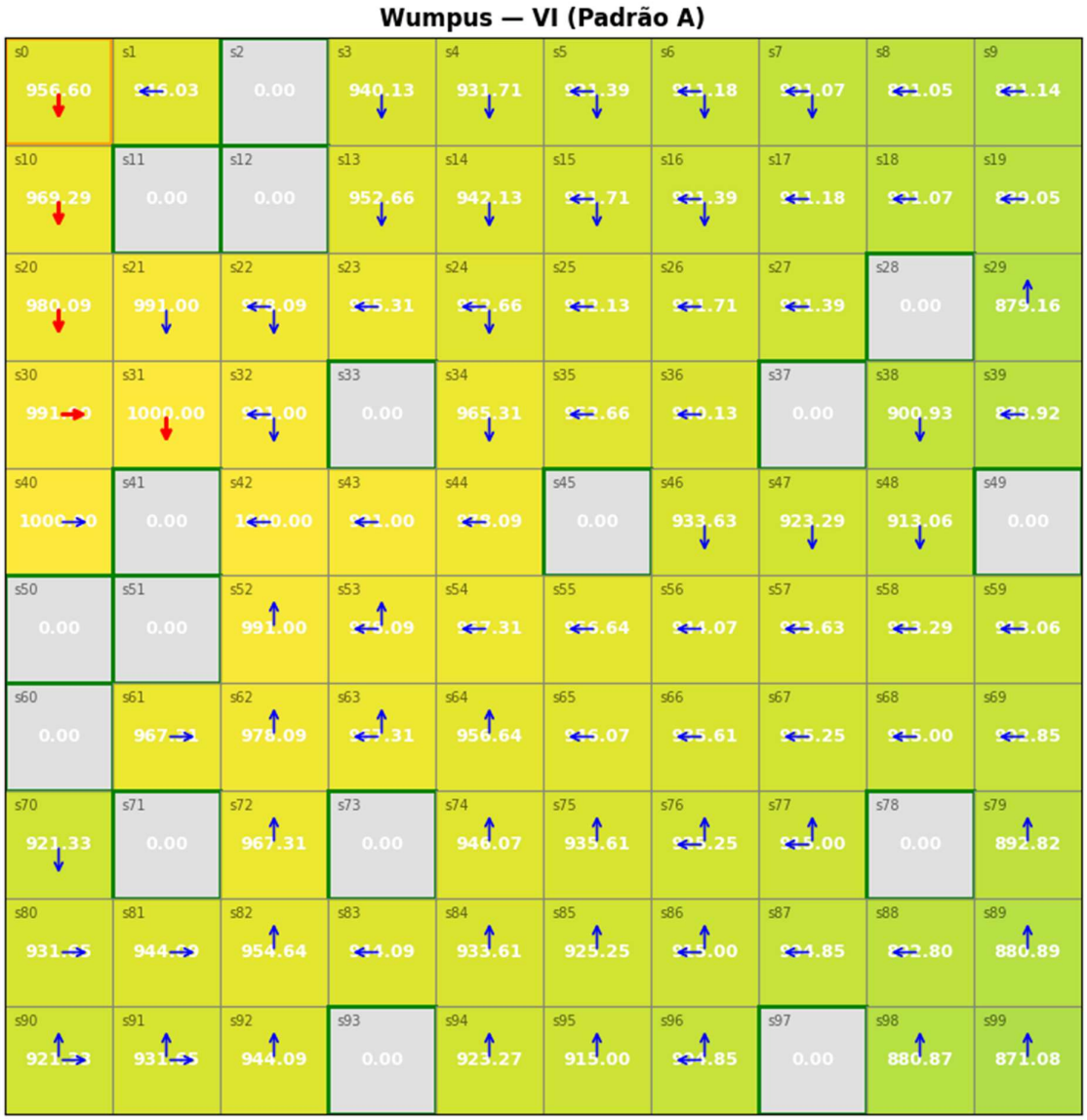



**Figura 20** – Resultados para a Etapa 1 do Jogo do Wumpus

A Figura 21 mostra as posições no início da Etapa 2.

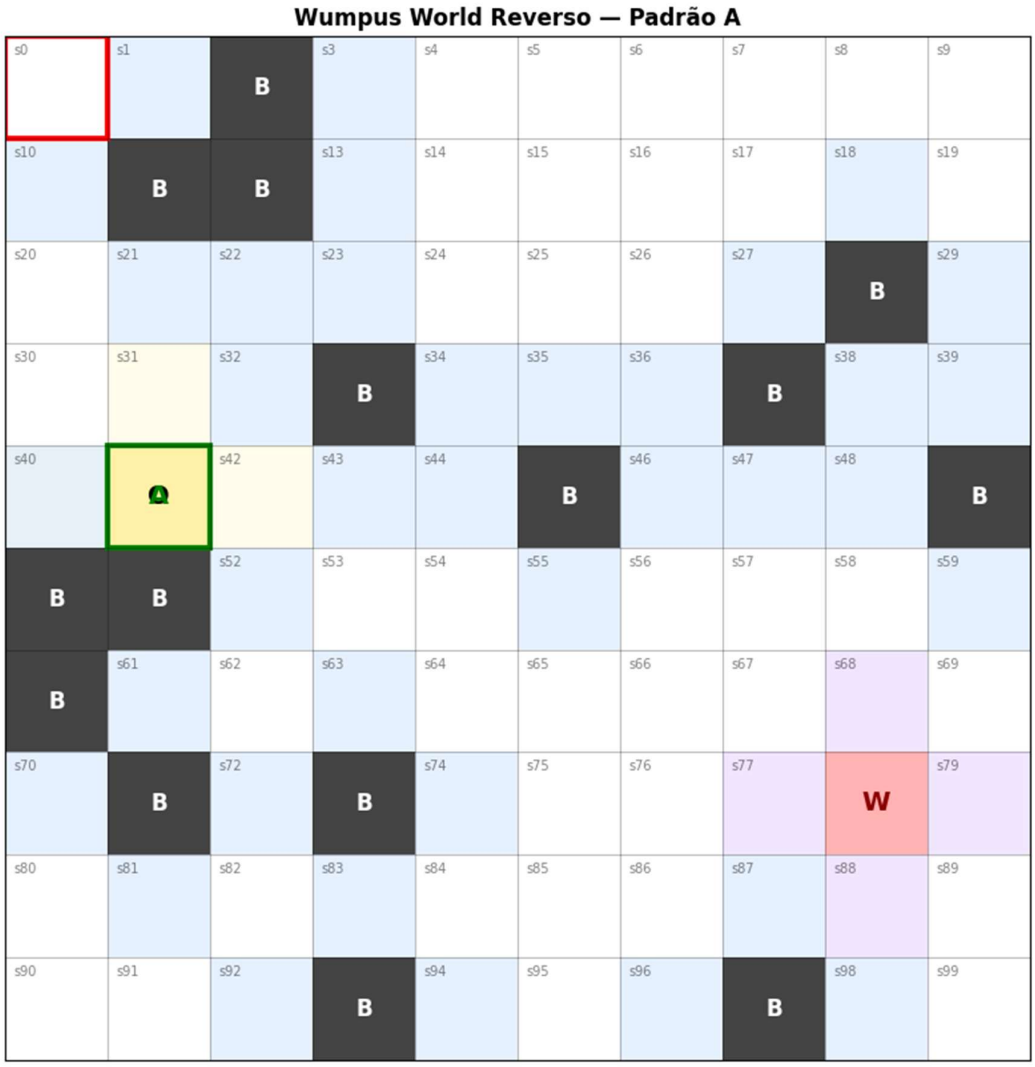



**Figura 21** – Posições no início da Etapa 2.

A Figura 22 apresenta os resultados para a Etapa 2 do jogo do Wumpus. Os valores de $V^*(s)$ são mostrados nas células do tabuleiro 10 x 10. As setas em vermelha indicam a Política Ótima $\pi^*(a \mid s)$ a partir do estado inicial.

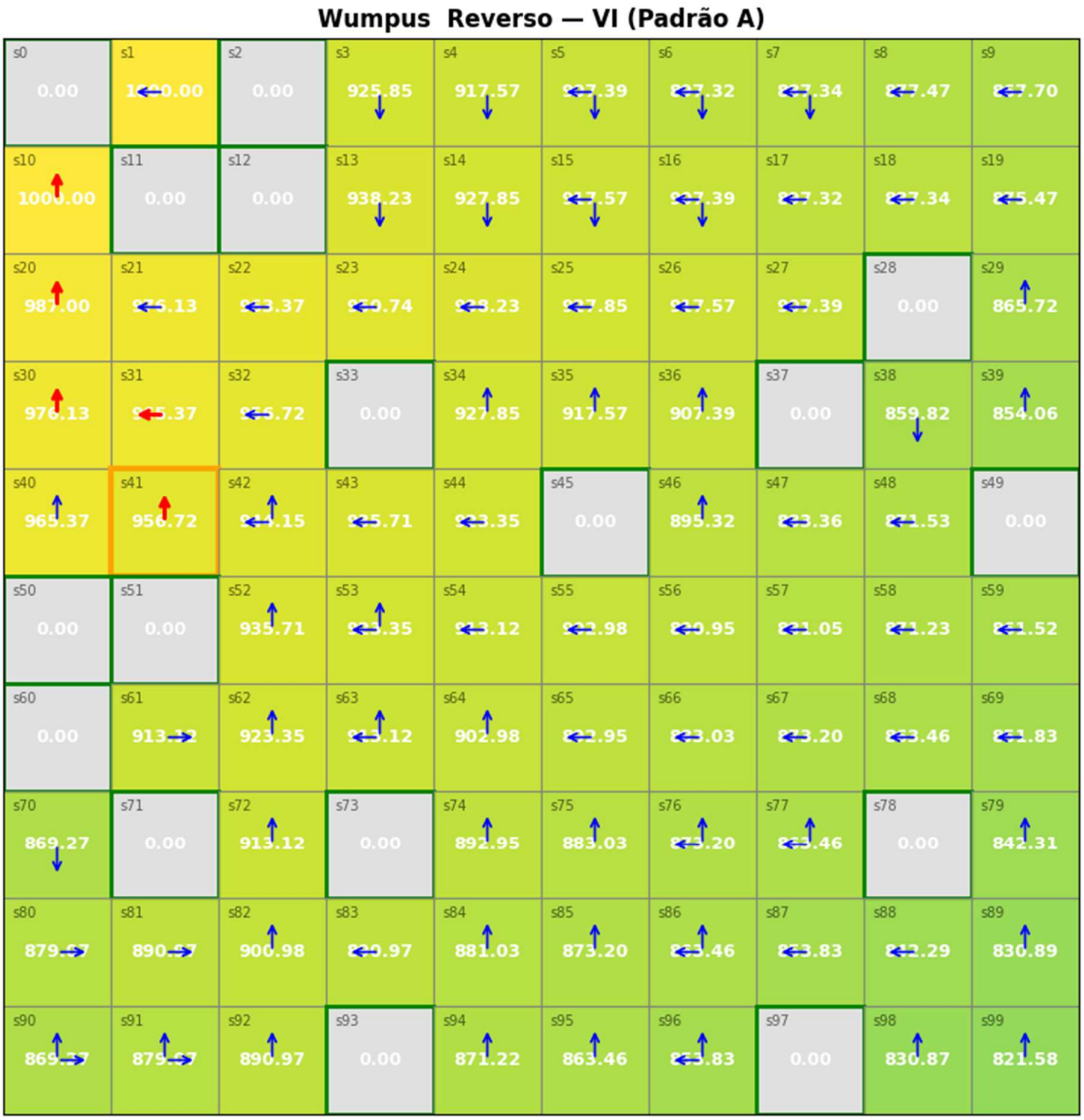



**Figura 22** – Resultados para a Etapa 2 do Jogo do Wumpus.

## 6. Conclusões:

- Os algoritmos de obtenção de política ótima para MDP funcionaram corretamente nos exemplos de jogos de tabuleiro apresentados. Os algoritmos VI e MPI para k = 1 apresentaram melhor desempenho em tempo de execução, sendo o PI bem mais pesado (ver Tabela 4). Observa-se no MPI que é desnecessária a convergência da função de valor de estado V(s) para começar a melhorar a política π(a|s), o que possibilita melhor desempenho em tempo de execução ao se ajustar o hiperparâmetro k.
- Os exemplos estudados possuem número de estados limitado (máximo de 100). Para problemas mais complexos, o número de estados impacta de forma relevante o tempo de computação — explosão de estados (ver Tabela 1). No próprio exemplo do Wumpus,

se considerássemos os subestados de percepções de vento, cheiro e luz em cada estado, em vez de tratá-los como recompensas, teríamos um número maior de estados.

- Uma limitação dos MDP é a necessidade de conhecer a priori todo o modelo da dinâmica P(r, s'|s, a). Nos jogos de tabuleiro estudados, essa dinâmica foi construída a partir das regras do jogo e do modelo de recompensas. Porém, isso nem sempre é possível em problemas reais.
- Como em muitos problemas da vida real o modelo não é conhecido, foram desenvolvidos algoritmos livres de modelo (Model Free), como SARSA e Q-Learning. No exercício do Wumpus, se o monstro também se mover a cada jogada, a dinâmica se torna mais complexa e o espaço de estados cresce, cenário em que um algoritmo livre de modelo se aplicaria.
- Alguns algoritmos baseiam-se em amostragem e busca em árvore com *pruning* para mitigar a explosão de estados, como Monte Carlo e MCTS. Além disso, os algoritmos de MDP requerem a observação dos estados do ambiente, o que muitas vezes não é possível; para esses casos foram desenvolvidos algoritmos para estados parcialmente observáveis, os POMDPs. Nesses problemas, as observações, por si só, não satisfazem a propriedade de Markov, o que impede o uso direto da Equação de Bellman sobre elas. A solução padrão reformula o problema no espaço de **estados de crença** (*belief states*) — distribuições de probabilidade sobre os estados ocultos —, onde a propriedade de Markov é restaurada e a Equação de Bellman volta a ser aplicável [9].
- Outras limitações ao uso da Equação de Bellman em problemas reais são mencionadas em [9]: a **não estacionaridade**, em que a dinâmica do ambiente varia com o tempo — como no exemplo do Wumpus, se o monstro também se mover a cada passo do jogador; a **restrição a recompensas escalares**, pois o formalismo padrão exige que o objetivo seja expresso como um único escalar, o que não contempla problemas com múltiplos objetivos conflitantes (que exigiriam vetores de recompensa); a **não aditividade das recompensas**, já que a Equação de Bellman pressupõe que o retorno é a soma descontada das recompensas, suposição que falha quando o agente tem preferências não lineares sobre a distribuição de retornos; e a presença de **múltiplos agentes** atuando no mesmo ambiente, caso em que o processo deixa de ser um MDP e passa a ser um jogo de Markov, no qual as transições e recompensas dependem da ação conjunta dos agentes e o conceito relevante passa a ser o de equilíbrio, não o de política ótima de agente único.
- No problema do Wumpus, utilizamos uma função de recompensa adaptada de [7]. Projetar uma função de recompensa para um problema específico não é trivial e geralmente exige experimentação. A referência [10] aborda essa questão e apresenta abordagens para lidar com ela.

## Referências


[1] [Sutton & Barto] *Reinforcement Learning – Second Edition* – The MIT Press - Capítulos 3 e 4 - 2018

[2] [Lapam] *Deep Reinforcement Learning Hands-On -Third Edition* – Capítulo 2 – 2024

[3] [Puttermann] Markov Decision Processes and Reinforcement Learning - https://github.com/martyput/MDP_book, acesso em 12/08/2026

[4] [Falcão] Da Teoria ao Código: Implementando *K-Armed Bandits* com Padrões de Projeto, 10/08/2026, https://github.com/kwahcarioca/Reinforcement-Learning/tree/main/K-bandidos/paper

[5] [AnáliseMacro] Cadeias de Markov no R: Prevendo o Mercado Financeiro – 2022 – acesso em 08/09/2026 - https://analisemacro.com.br/mercado-financeiro/aplicacao-de-cadeias-de-markov-no-mercado-financeiro-com-o-r/

[6] [Bellman] *Dynamic Programming – Rand Corporation* -1957

[7] [Russel, Norvig] Inteligência Artificial – Uma Abordagem Moderna – Quarta Edição – Capítulos 7 e 17, LTC – 2022

[8] [Gamma e outros] – *Design Patterns – Elements of Reusable Object-Oriented Software* – Addison-Wesley, 1995

[9] [Pineau] *Beyond's Bellman's Legacy – Rethinking what we value – Reinforcement Learning Conference - Edmonton* – AMII – 2025 - https://www.youtube.com/watch?v=PZcaMKxljxc - acesso em 17/09/2026 -

[10] [Muslimani] *Reward Design and Evaluation in Reinforcement Learning – AI Seminar* – AMII – 2026 - acesso em 17/09/2026 - https://www.youtube.com/watch?v=Mva478tO6s4&list=PLKlhhkvvU8-aXXTPoxjxLWXySum1SjLH0&index=16&t=645s

[11] GitHub - https://github.com/kwahcarioca/Reinforcement-Learning/tree/main/MDP/notebooks